UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

Semester 01 2025/2026

Subject : Database Programming (SECP3623)
Section : Section 02
Task : Project II
Due : 12th January 2026 (10%)

Name	Matric No.
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	A23CS0117
IMAN ABADI BIN MOHD NIZWAN	A23CS0084
MOHAMED ALIF FATHI BIN ABDUL LATIF	A23CS0112

Presentation Link: https://youtu.be/wr6CDK_ZpJc

TABLE OF CONTENTS

1.0 Introduction

1.1 Background and Objectives

BACKGROUND

A Library Management System is responsible for handling various library operations such as managing books, tracking borrowing activities, maintaining student records, and monitoring fines on students. Traditional relational databases can sometimes struggle with scalability, changing data structures, and handling complex relationships efficiently. MongoDB, as a NoSQL document-based database, provides flexible schema design, high scalability, and efficient data handling for large and dynamic datasets.

In this project, MongoDB is used to store and manage library related data such as books, users, book loans, and fines enabling faster data access and better performance for library operations.

OBJECTIVES

1. To implement MongoDB as a database solution for the Library Management System to manage book, member, and borrowing information efficiently.
2. To demonstrate CRUD operations (Create, Read, Update, Delete) using MongoDB for handling library data such as adding books, searching books, updating records, and deleting outdated records.
3. To utilize MongoDB features such as arrays, embedded documents, and references to represent real-world relationships like borrowed books and user information.
4. To perform query operations and projections to retrieve only relevant data, improving performance and data retrieval efficiency.
5. To provide a scalable and flexible data structure that can support future library expansion such as additional book categories, digital materials, and user management enhancements.

1.2 Team Members and Assigned Roles

1. MUHAMMAD AFIQ DANIAL BIN ROZAIDIE A23CS0117
    - I contribute to CRUD implementation like insertion, query, update and also deletion.
    - I suggest the real world domain which is Library Management System.
    - Make a background and objectives of the report
2. IMAN ABADI BIN MOHD NIZWAN A23CS0084
    - Provide explanation on query performance speed from applying indexing
    - Did research on NoSQL's security and limitations, covering basic access control, injection risks and its limitations.
    - Concluded the whole report, summarizing what was learned and suggesting improvement.
3. MOHAMED ALIF FATHI BIN ABDUL LATIF A23CS0112
    - Contributes to NoSQL data model part in reporting.
    - Contributes to advanced operations which are index creation, aggregation pipeline and sorting demonstrations.

2.0 NoSQL Data Model

This system uses MongoDB, a document-oriented NoSQL database. For our project, the database was created named LibraryDB. Data is stored in collections with a flexible JSON file of documents, making it suitable for a library management system where data structures may be updated from time to time.

2.1 Collections

Users Collection

The users collection stores information about library members who are allowed to borrow books. It includes personal details such as name, email, contact number, and address. This collection helps identify borrowers and maintain their borrowing records within the system.

Books Collection

The books collection contains details of all books available in the library. It stores information such as book title, authors, categories, and publisher details. This collection is used to manage the library's book catalog and support book search and loan operations.

Loans Collection

The loans collection records each borrowing transaction between a user and a book. It stores references to the user and the borrowed book, along with loan dates, due dates, return dates, and loan status. This collection is essential for tracking active and completed book loans.

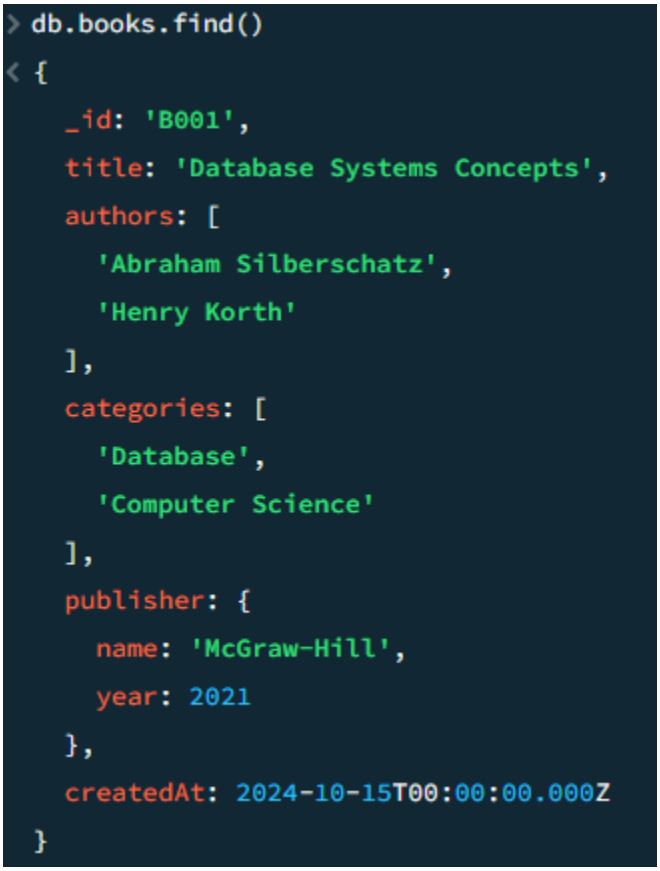
Fines Collection

The fines collection stores information related to penalties imposed on users for late book returns. It includes the fine amount, reason for the fine, payment status, and references to the related loan and user. This collection helps the library manage overdue charges and payment tracking.

2.2 JSON Documents

Users Collection

Books Collection



```
> db.books.find()
< {
    _id: 'B001',
    title: 'Database Systems Concepts',
    authors: [
      'Abraham Silberschatz',
      'Henry Korth'
    ],
    categories: [
      'Database',
      'Computer Science'
    ],
    publisher: {
      name: 'McGraw-Hill',
      year: 2021
    },
    createdAt: 2024-10-15T00:00:00.000Z
  }
```

Loans Collection

```
> db.loans.find()
< {
    _id: 'L001',
    userId: 'A23CS0001',
    bookId: 'B001',
    loanDate: 2024-12-01T00:00:00.000Z,
    dueDate: 2024-12-14T00:00:00.000Z,
    returnDate: null,
    status: 'BORROWED'
  }
```

Fines Collection

```
> db.fines.find()
< {
    _id: 'F001',
    loanId: 'L001',
    userId: 'A23CS0001',
    amount: 5,
    reason: 'Late return',
    payment: {
      status: 'UNPAID',
      method: null
    },
    issuedDate: 2024-12-16T00:00:00.000Z
  }
```

2.3 Embedding vs Referencing

In MongoDB, data relationships can be modeled using embedding or referencing, depending on how the data is accessed and updated. The library management system uses a combination of both approaches to achieve better performance and data consistency.

Embedding

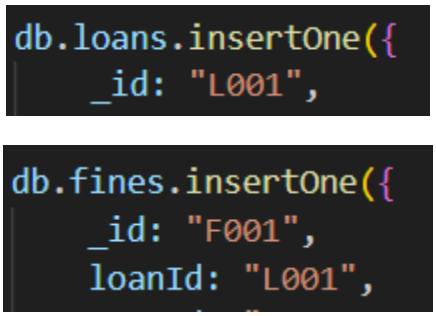
Embedding involves storing related data within the same document as a nested or embedded document. This approach is suitable when the embedded data is tightly coupled with the main document and is frequently accessed together. The picture below shows the usage of embedding in this system.

```
address: {
    street: "123 Jalan Melati",
    city: "Muar",
    state: "Johor",
},
```

In the library management system, embedding is used for attributes such as address in the users collection, publisher in the books collection, and payment details in the fines collection. These data elements are specific to their parent document and are not shared with other collections. By embedding this information, the system can retrieve all relevant data in a single query, improving read performance and reducing the need for complex joins.

Referencing

Referencing stores related data in separate collections and connects them using identifiers. This approach is useful when data is shared among multiple documents.



In this system, referencing is applied in the loans and fines collections. For example, userId in the loans collection references the users collection, while bookId references the books collection. Similarly, loanId and userId in the fines collection reference the corresponding loan and user records. This design avoids data duplication and ensures data consistency when user or book information is updated.

2.4 Data Types

Collection	Data Type	Attributes
Books	String	_id, title, authors, categories, publisher.name
	Number	publisher.year
	Date	createdAt
Users	String	_id, name, email, contact, address.street, address.city, address.state, borrowHistory
	Date	createdAt
Loans	String	_id, userId, bookId, status
	Date	loanDate, dueDate, returnDate
Fines	String	_id, loanId, userId, reason, payment.status, payment.method
	Number	amount
	Date	issuedDate

3.0 CRUD Implementation Summary

3.1 Explanation of insert, query, update, delete operations

1. Insert()

The insert() method is a fundamental operation in MongoDB that is used to add new documents to a collection. This method is flexible, allowing developers to insert either a single document or multiple documents in a single operation, which can significantly enhance performance and reduce the number of database calls. For example, the insert method is db.collection.insert({...}).

2. Query/Read

MongoDB query operators are used to filter documents based on specific conditions. These operators compare values, match patterns, handle arrays, and process geospatial data. For example, the query operation is db.collection.findOne({...}).

3. Update()

The MongoDB update() method is a method that is used to update a single document or multiple documents in the collection. When the document is updated the _id field remains unchanged. The db.collection.update({...}) method updates a single document by default.

4. Delete()

The delete method is used to delete both single or multiple documents in MongoDB. It removes the document that matches the specified filter criteria. deleteOne() operation is used to delete a single document while deleteMany() operation is used to delete multiple documents at once.

3.2 Screenshots of MongoDB Compass or shell execution

1. Insert Operation

- insertOne()

```
// Insert a new user into the users collection using insertOne()
db.users.insertOne({
    _id: "A23CS0002",
    name: "Danial Afiq",
    email: "dani@example.com",
    contact: "0198765432",
    address: {
        street: "456 Jalan Kenanga",
        city: "Kuala Lumpur",
        state: "Selangor"
    },
    borrowHistory: ["B001", "B004"],
    createdAt: new Date("2024-12-01")
})
```

This insertOne() operation can only insert one new document at one time.

- insertMany()

```
// Insert multiple users into the users collection using insertMany()
db.users.insertMany([{
    _id: "A23CS0003",
    name: "Fathi Alif",
    email: "fathi@example.com",
    contact: "0198765433",
    address: {
        street: "789 Jalan Seri",
        city: "Kuching",
        state: "Sarawak"
    },
    borrowHistory: ["B003", "B006"],
    createdAt: new Date("2024-12-01")
},
{
    _id: "A23CS0004",
    name: "Abadi Iman",
    email: "abadi@example.com",
    contact: "0198767434",
    address: {
        street: "101 Jalan Seri",
        city: "Kota Bharu",
        state: "Kelantan"
    },
    borrowHistory: ["B002"],
    createdAt: new Date("2024-12-01")
}])
```

The insertMany() method in MongoDB adds multiple documents to a collection at once.

2. Update Operation

- $set

```
// Update loan with _id "L004" to change bookId, loanDate, dueDate, and status using $set
db.loans.updateOne({_id: "L004"},
    {$set: {
        bookId: "B004",
        loanDate: new Date("2024-12-12"),
        dueDate: new Date("2024-12-26"),
        status: "BORROWED"
    }}
)
```

$set operator is a powerful update operator used to add new fields to a document or modify the value of existing fields without overwriting the entire document. It is the primary way to perform partial updates.

- $inc

```
// Increase fine amount by 2.0 for all fines with payment status "UNPAID" using $inc
db.fines.updateMany({"payment.status": "UNPAID"},
    {$inc: {amount: 2.0}}
)
```

$inc operator is used to atomically increment or decrement the value of a field by a specified amount.

- $push

```
// Add a bookId to the borrowHistory array of user with _id "A23CS0004" using $push
db.users.updateOne({_id: "A23CS0004"},
    {$push: {borrowHistory: "B005"}}
)
```

$push operator appends a specified value to an array field within a document. If the field does not exist, $push will create it as a new array field.

- $pull

```
// Remove "Clifford Stein" from the authors array of the book titled "Introduction to Algorithms" using $pull
db.books.updateOne({title: "Introduction to Algorithms"},
    {$pull: {authors: "Clifford Stein"}}
)
```

$pull operator is used to remove all instances of a specified value or values that match a certain condition from an existing array within a document.

3. Delete Operation

- deleteOne()

```
// Delete the fine with _id "F003"
db.fines.deleteOne({_id: "F003"})
```

deleteOne() operation removes at most a single document from a collection that matches the specified filter. If multiple documents match the criteria, only the first one encountered is deleted.

- deleteMany()

```
// Delete all fines with payment status "PAID"
db.fines.deleteMany({"payment.status": "PAID"})
```

The deleteMany() operation in MongoDB is used to remove all documents from a collection that match a specified filter.

3.3 Projections and query tests

- findOne()

```
// Find a user by name and retrieve specific fields using findOne()
db.users.findOne({name: "Danial Afiq"}, {_id: 1, email: 1, contact: 1})
```

Output:

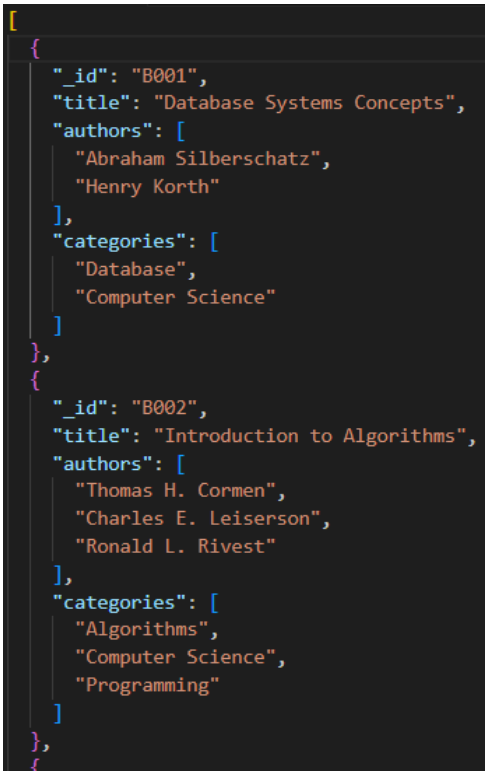
```
{
  "_id": "A23CS0002",
  "email": "dani@example.com",
  "contact": "0198765432"
}
```

The findOne() operation is a method used to retrieve a single document that matches specified criteria from a collection.

- find() and Projection

```
// Find all books and retrieve their titles, authors, and categories using find()
db.books.find({}, {title: 1, authors: 1, categories: 1})
```

Output:



The find() method returns a cursor to the matching one document or more documents. While the projection is used to specify which fields should be included in the query result. In this example, the query retrieves the all books document but only returns the _id, title, authors and categories fields instead of the entire document.

- $gt and $lt

```
// Find fines with amount greater than 5.0 and less than 25.0 using $gt and $lt operators
db.fines.find({amount: {$gt: 5.0, $lt: 25.0}},
    {loanId: 1 ,amount: 1, reason: 1}).sort({amount: 1})
```

Output:

```
[
  {
    "_id": "F001",
    "loanId": "L001",
    "amount": 7,
    "reason": "Late return"
  },
  {
    "_id": "F004",
    "loanId": "L005",
    "amount": 24.5,
    "reason": "Lost book"
  }
]
```


The $gt ("greater than") and $lt ("less than") operators are comparison query operators used to filter documents based on a field's value relative to a specified value. They can be applied to numbers, strings, and dates.

- $eq

```
// Find fines with amount equal to RM 0.0 using $eq operator
db.fines.find({amount: {$eq: 0.0}})
```

Output:


```
[
  {
    Edit Document
    "_id": "F003",
    "loanId": "L003",
    "userId": "A23CS0003",
    "amount": 0,
    "reason": "No fine",
    "payment": {
      "status": null,
      "method": null
    },
    "issuedDate": {
      "$date": "2024-12-08T00:00:00Z"
    }
  }
]
```


$eq operator (equality operator) is a comparison query operator that selects documents where the value of a specified field is exactly equal to a given value.

- $in

```
// Find loans with status either "BORROWED" or "PENDING" using $in operator
db.loans.find({status: {$in: ["BORROWED", "PENDING"]}}, {userId: 1, bookId: 1, status: 1})
```

Output:

$in operator is a comparison query operator that selects documents where the value of a field equals any value in a specified array. It is used to query for multiple possible values for a field in a single, concise query.

- $and

```
// Find books that belong to both "Computer Science" and "Database" categories using $and operator
db.books.find({$and: [{categories: "Computer Science"}, {categories: "Database"}]},
    {title: 1, authors: 1, categories: 1})
```

Output:

```
[
  {
    "_id": "B001",
    "title": "Database Systems Concepts",
    "authors": [
      "Abraham Silberschatz",
      "Henry Korth"
    ],
    "categories": [
      "Database",
      "Computer Science"
    ]
  },
  {
    "_id": "B006",
    "title": "Introduction to Database Systems",
    "authors": [
      "Abraham Silberschatz",
      "Henry F. Korth",
      "S. Sudarshan"
    ],
    "categories": [
      "Database",
      "Computer Science"
    ]
  }
]
```


$and operator performs a logical AND operation on an array of one or more expressions and selects only those documents that satisfy all the specified conditions.

- $or

```
// Find books that are either in "Programming" category or published in the year 2020 using $or operator
db.books.find({$or: [{categories: "Programming"}, {"publisher.year": 2020}]},
    {title: 1, authors: 1, categories: 1, publisher: 1})
```

Output:

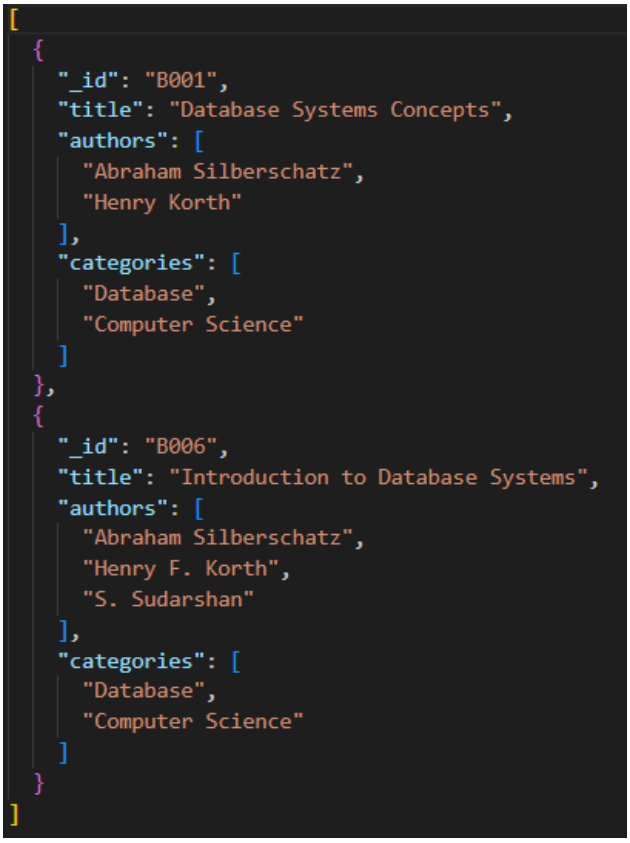
```
[
  {
    "_id": "B002",
    "title": "Introduction to Algorithms",
    "authors": [
      "Thomas H. Cormen",
      "Charles E. Leiserson",
      "Ronald L. Rivest",
      "Clifford Stein"
    ],
    "categories": [
      "Algorithms",
      "Computer Science",
      "Programming"
    ],
    "publisher": {
      "name": "MIT Press",
      "year": 2022
    }
  },
  {
    "_id": "B003",
    "title": "Advanced Mathematics",
    "authors": [
      "Richard G. Brown"
    ],
    "categories": [
      "Mathematics",
      "Advanced Studies"
    ],
    "publisher": {
      "name": "Springer",
      "year": 2020
    }
  },
```


$or operator performs a logical OR operation on an array of two or more expressions and selects the documents that satisfy at least one of the specified conditions.

- Array Queries

```
// Find books authored by "Abraham Silberschatz" in array query
db.books.find({authors: "Abraham Silberschatz"}, {title: 1, authors: 1, categories: 1})
```

Output:



Array query operations are used to search and retrieve documents where the values inside an array field match certain specified conditions.

4.0 Advanced Operations

4.1 Index Creation

Single Field Index

```
db.users.createIndex({ name: 1 })
```

A single-field index on name in users helps quickly retrieve users in alphabetical order as the index already maintains sorted data for efficient sorting operations.

Compound Index

```
db.fines.createIndex({ userId: 1, "payment.status": 1 })
```

A compound index on userId and payment.status in fines allows efficient retrieval of unpaid fines for specific users as admin often need to check the unpaid fines of a user.

4.2 Aggregation Pipeline

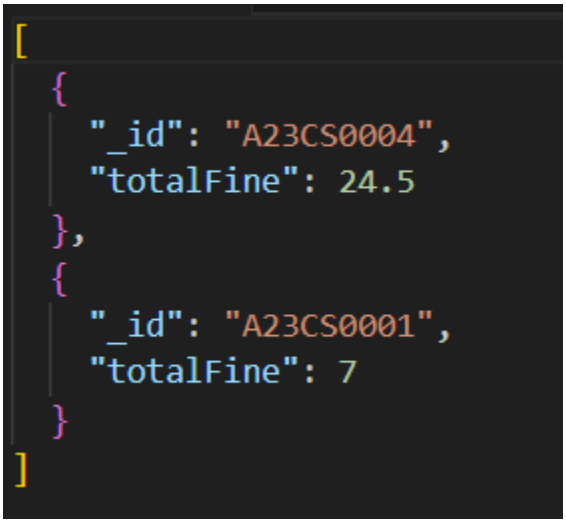
Total number of books

```
db.books.aggregate([{ $count: "totalBooks" }])
```

Counts the total number of documents in the books collection.

Total unpaid fines per user

```
db.fines.aggregate([{ $match: { "payment.status": "UNPAID" } },
{ $group: { _id: "$userId", totalFine: { $sum: "$amount" } } },
{ $sort: { totalFine: -1 } }])
```



Filters fines to only include unpaid status, groups by userId, calculates the sum of fine amounts and sorts results in descending order by total fine amount.

Total books per category

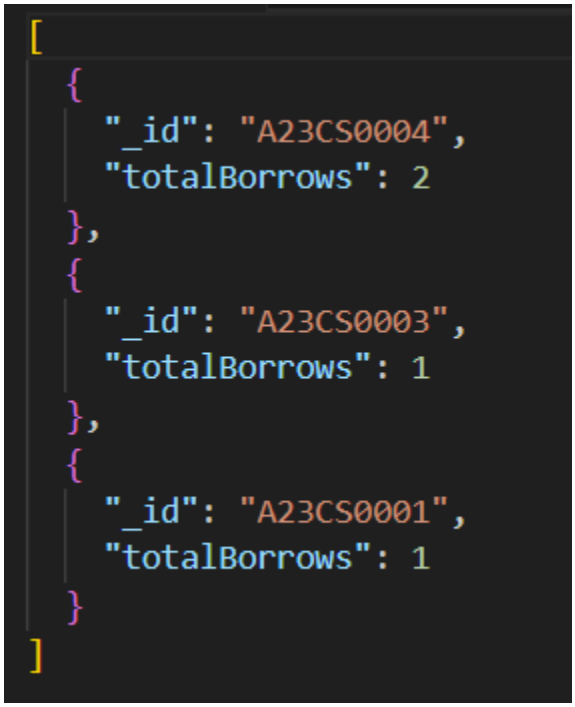
```
db.books.aggregate([{ $unwind: "$categories" },
{ $group: { _id: "$categories", totalBooks: { $sum: 1 } } },
{ $sort: { _id: 1 } }])
```


```
[
  {
    "_id": "Advanced Studies",
    "totalBooks": 1
  },
  {
    "_id": "Algorithms",
    "totalBooks": 1
  },
  {
    "_id": "Computer Science",
    "totalBooks": 4
  },
  {
    "_id": "Database",
    "totalBooks": 2
  },
  {
    "_id": "Mathematics",
    "totalBooks": 1
  },
  {
    "_id": "Programming",
    "totalBooks": 1
  },
  {
    "_id": "Psychology",
    "totalBooks": 1
  },
  {
    "_id": "Self-Help",
    "totalBooks": 1
  },
  {
    "_id": "Software Engineering",
    "totalBooks": 1
  }
]
```

Unwinds the categories array to create a document for each category, groups by category and sums the total number of books for each category.

Top 3 users with the highest number of books borrowed

```
db.loans.aggregate([{ $group: { _id: "$bookId", totalBorrows: { $sum: 1 } } },
{ $sort: { totalBorrows: -1 } },
{ $limit: 3 }])
```



Groups loans by userId, counts the number of times each user has borrowed a book, and sorts results in descending order by the count.

4.3 Sorting

To organize search results in MongoDB, use the .sort() method with a number of rules like use 1 for ascending order and -1 for descending order. This allows the system to quickly arrange data ascending or descending based on users' needs. Below are examples of sorting usage.

Ascending

```
db.users.find().sort({ name: 1 })
```

Sort users by name in ascending order using .sort()

Descending

```
db.books.find().sort({ "publisher.year": -1 })
```

Sort books by published year in descending order using .sort()

4.4 Performance Improvements

Indexing methods increase query speed by establishing a distinct data structure that enables the database engine to discover and retrieve certain data rows more rapidly. Instead of scanning the whole document, the database may utilize the index to rapidly locate the relevant data. This improvement is especially useful for complicated queries with huge datasets, since it may significantly reduce the time necessary for data retrieval processes.

5.0 Security & Limitations

5.1 Basic Access Control

Access control is one of the most important security features in system development and management as it involves how users access online resources and the rights they have over those resources. It ensures that only authorized users or applications can access, modify or manage data. There are several access control mechanisms used by NoSQL systems, two of which are role-based access control (RBAC) and attribute-based access control (ABAC).RBAC is a security technique built on the principle of least privilege. User privileges are the user's right to perform a particular action or to perform an action on any object of a particular type. A user within the system can have one or more roles, where each role can have multiple privileges and capabilities based on the business requirements. Large businesses often use this mechanism because it is simple to implement and scalable. Each role can be defined with specific permissions, and then users are assigned to them, making it simple to manage access as the business expands.ABAC on the other hand, dynamically regulates access to users by utilizing the idea of attributes, which are name-value pairs that describe the statuses of users, objects, and the environment. This enables ABAC to model complicated policies and is quite flexible. It can also grant access to external ad hoc users by authenticating characteristics rather than credentials.

5.2 Injection Risk

NoSQL injection is a security flaw in which attackers can modify NoSQL database queries by inserting malicious input, exploiting the flexibility of NoSQL's security structures. NoSQL databases use JSON-like syntax for queries and allow dynamic query construction. In insecure implementations, taking user input directly into these queries has the potential to influence database behavior. Common exploitation techniques include Boolean-based injection by injecting queries like {"$ne": null} into the login credentials input, like password and username to bypass authentication checks. To prevent these vulnerabilities, it is recommended to sanitize and validate user inputs by excluding special characters that can be interpreted as operators.

5.3 Database Constraints or Limitations

While NoSQL databases are advantageous in terms of scalability and flexible schemas. There are still limitations that need to be considered. Firstly, there is the lack of standardization where the NoSQL database uses its own unique schema. They have their own query language or API, which could make it harder for developers to learn. It can also make it difficult to integrate with other systems. Moreover, NoSQL has weaker data consistency and ACID implementation, as it prioritizes scalability instead. This leads to eventual consistency, where updated nodes are not reflected immediately on other nodes, making it unsuitable for high transaction systems. Other than that, it lacks complex query flexibility and complex joins. Unlike SQL, querying related data needs to be done manually, which slows down queries, especially analytical queries. The alternative to that is by denormalizing data, making redundant copies of data in order to speed up queries at the cost of increased maintenance complexity.

6.0 Teamwork

Team Member	What we have learned
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	I learned the basic operations of NoSQL(MongoDB) which are CRUD operations specifically insert, query, update and delete. These basic techniques are enough for me to gain practical skills in creating, reading, updating, and deleting documents in a database, which helps me manage and manipulate data efficiently in real-world applications.
IMAN ABADI BIN MOHD NIZWAN	I learned the basics of NoSQL systems, including their security vulnerabilities and limitations compared to SQL-based databases. This can help me in choosing the types of database that I can use that are most suitable for my project scope since I know the strengths and weaknesses of both of them

MOHAMED ALIF FATHI BIN ABDUL LATIF	I learned how to use aggregation, indexing, and sorting in MongoDB. Aggregation helped me summarize data, such as calculating total fines and grouping records. Sorting helped arrange data in ascending and descending order to make it easier to read. Indexing showed me how queries can run faster by reducing data scanning.

7.0 Conclusion

Through this project, various NoSQL concepts were implemented based on real-world scenarios through practical applications. Building the NoSQL data architecture helped in learning data arrangements using collections and JSON documents rather than traditional relational tables. The comparison of embedding and referencing helped in understanding the right balance between performance, data redundancy, and maintainability. The CRUD aggregation pipelines strengthened practical skills in performing fundamental operations using MongoDB Compass and the MongoDB shell and helped with the understanding of indexing in improving performance using two indexing methods. Additional knowledge on how NoSQL is used in real-world settings is also gained through research on its security and limitations section of the report. A suggested improvement for this project is a practical demonstration of how indexing affects query performance through benchmarking and a real demonstration of how SQL injection negatively impacts the database behaviors.